

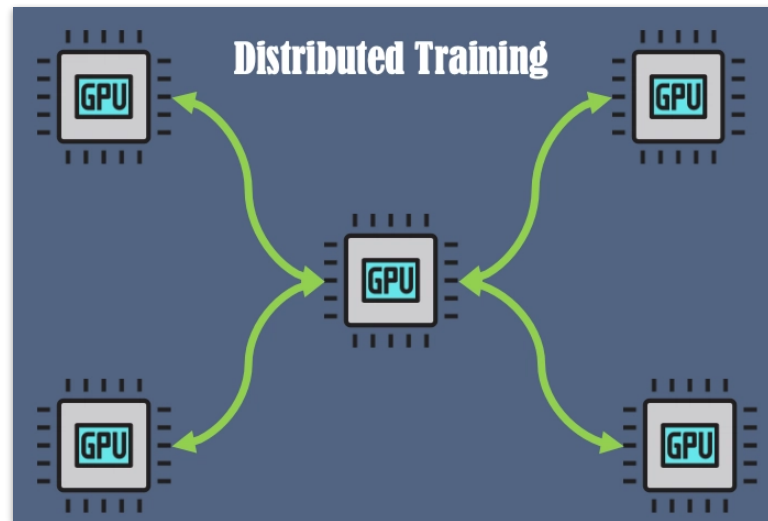


Fault-Tolerant Training and Verification

Priyanka Jakka, Riya Jain, Khush
Makadia, Mihir Shah, Anya Parekh

Problem

- Large Language Models and Deep Neural Networks are becoming increasingly large
- Distributed training requires multiple GPUs and is incredibly complex
- At scale, this incurs significant computational cost



Current Approach



Data Parallelism

Each worker has a copy of the full model, the input dataset is sharded, and workers aggregate their gradients periodically.



Tensor Parallelism

Splits tensors across multiple GPUs, a single operation is distributed across GPUs, high-speed communication required to synchronize results.



Pipeline Parallelism

Partitions by stages, each stage is assigned to a different GPU, data flows through stages like an assembly line.

Most large-scale training approaches combine all three techniques!



01

TrainVerify

Problems

- Parallelization is necessary for modern day training
- Bugs that occur during training are difficult to detect
 - Ex. A small scaling bug in MegatronLM bug took several months to find
- Floating point computation can introduce numerical inconsistencies
- Cannot verify results with a single device

Parallelization Bugs

- Improper slicing and uneven batch distribution
- Improper tensor partitioning and poorly transformed operators during forward/backward propagation
- Metrics like loss average or gradient norms can be inaccurate if tensor partitioning is incorrect
- Pipeline scheduling can misorder or misallocate

Cause	MegatronLM	DeepSpeed	nnScaler
op-transformation	16	18	19
scheduling	4	1	4
communication	8	13	5
Total	26	28	25

Table 1. *Parallelization-specific bugs* in distributed training systems, based on commit logs and GitHub issues (categories may overlap).

Naive Approach

Design a new training software stack that guarantees correctness from the start

External Dependencies
+ High Concurrencies

Incompatibility with
Training Libraries

Performance Loss

Important Definitions



Execution Plan

A representation of a full training iteration of the distributed model.



Logical Definition

Represented by a DFG, logical because operators and tensors are described mathematically.



Parallelization Equivalence

The execution plan is equivalent to the model's logical definition.



Lineage

Data structure that preserves the semantics of parallelization, indicating how a tensor in the original logical model is partitioned or replicated during parallelization.

Parallelization Equivalence

Parallelized model g is parallelization equivalent to a logical model f

$$f \text{ iff } \forall x \in \mathcal{X}, g(x) = f(x)$$

Logical DFG: $G_l := (V_l, E_l)$
Mapping:

$G_p := (V_p, E_p)$ **DFG:**

$$\mathcal{M} : E_l \rightarrow E_p^{\mathbb{N}}$$

G_p is parallelization equivalent to G_l iff:

$$\forall t \in E_l, \forall x \in \mathcal{X}, t(x) = \bigoplus_{t_i \in \mathcal{M}(t)} t_i(x)$$

Parallelization Equivalence Example

$$E_1 : g_y[\text{lower-half}] == g_{y2} + g_{y3}$$

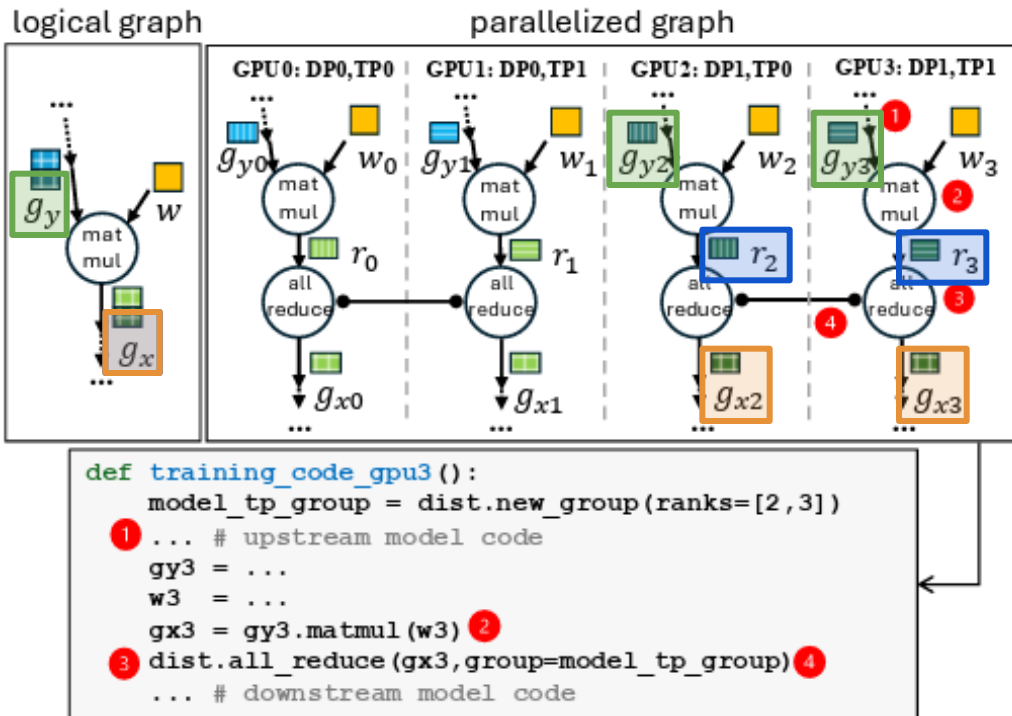
- Inconsistent tensor partition

$$E_2 : g_x[\text{lower-half}] == r_2 + r_3$$

- Incorrect computation operator parallelization

$$E_3 : g_x[\text{lower-half}] == g_{x2} == g_{x3}$$

- Missing or wrong communication operator
- Incorrect communication group



Research Questions

- RQ1 How do we formulate the verification problem?
- RQ2 What representation do we use to carry out the verification?
- RQ3 How do we scale the verification to large DNNs that have hundreds of billions of parameters?

Solution Overview



Leverage parallelization equivalence and establish correctness at the level of the execution plan

RQ1

TrainVerify

- Expose the DFG representation of the logical definition of a DNN
- Use the execution plan to formulate the parallelized DFG representation
- Use PE to compare inputs/outputs between both DFGs

TrainVerify Design

RQ3



Symbolic Data Flow Graph

TrainVerify converts logical model and its execution plan into symbolic DFGs and verifies the equivalence between these two representations.

RQ2



Shape Reduction

Mitigates scale explosion in LLMs by reducing tensor shapes while preserving their structural and functional properties.



Staged Verification

Partition DFGs into stages, each treated as a separate verification unit and verified concurrently. These units are chained into an end-to-end proof by verifying input-output equivalence for dependent units.

Symbolic Data Flow Graphs (sDFGs)

- More articulate than off-the-shelf DFGs (backpropagation, optimizers, metrics, etc. included)
- Includes formal mathematical definitions compatible with SMT solvers.
- Replaces DFG nodes & edges with symbolic operators & tensors

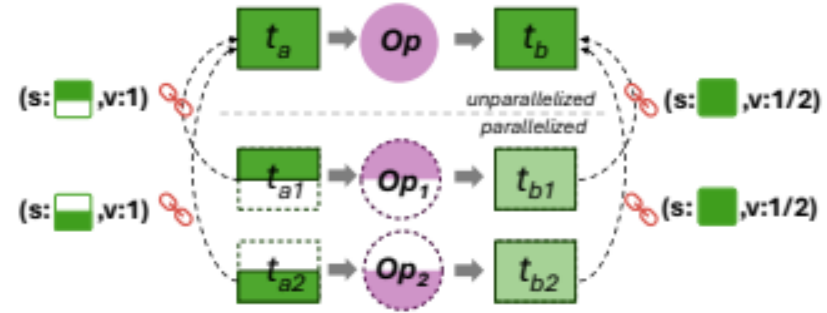


Figure 4. Lineage of tensors.

Lineage: Data structure used to preserve parallelization semantics. Used to determine how partitioning and replicating from parallelization affects tensors and inputs/outputs.

Shape Reduction

$$c_{1,1} = a_{1,1} \cdot b_{1,1} + a_{1,2} \cdot b_{2,1} + a_{1,3} \cdot b_{3,1}$$

$$c_{2,2} = a_{2,1} \cdot b_{1,2} + a_{2,2} \cdot b_{2,2} + a_{2,3} \cdot b_{3,2}$$

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,1} & b_{3,2} & b_{3,3} \end{bmatrix} = \begin{bmatrix} c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix}$$

Problem: Many operations have redundancy that arise from the SIMD property inherent to DNN operators. Redundant elements can be omitted without compromising the verification outcome.

Algorithm

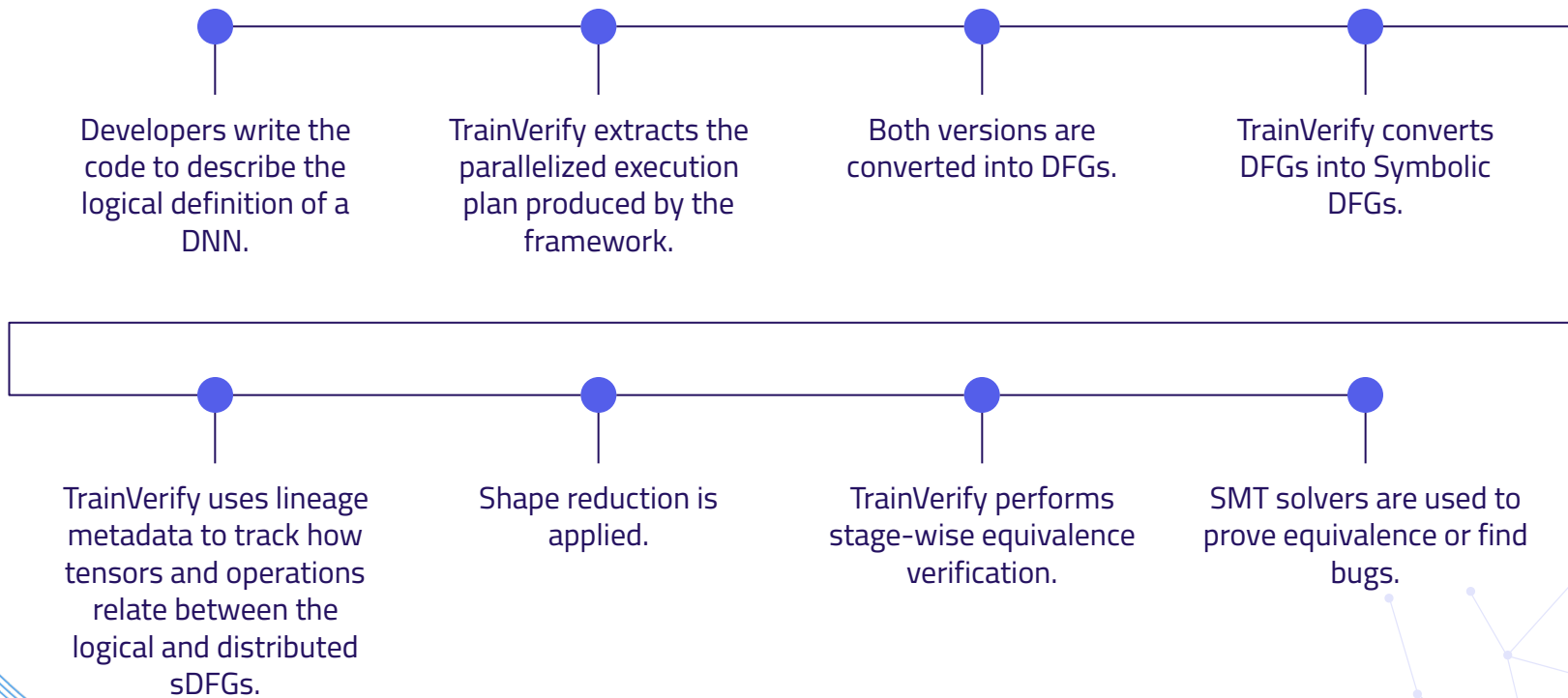
- Topologically sort the input
 - Calculate the minimum shape abiding these shape constraints:
1. Min. size: 1 – original size
 2. Enforce shape alignment
 3. Enforce semantics
 4. Minimize total tensor volume

Staged Verification

Problem: As model depth increases, symbolic expressions become intractable often causing solvers to return unknown/fail to terminate due to memory overflow.

1. Logical DFG and multi-device DFG are aligned and partitioned into stages
 - TrainVerify determines stages by repeatedly applying backward slicing on the two graphs, which is aided by tensor alignment to guide the partitioning
1. Pool of workers concurrently verifies each subgraph's stage equivalence
2. If all stages pass the verification, the process guarantees end-to-end equivalence between the two graphs

TrainVerify Overview



TrainVerify System Overview

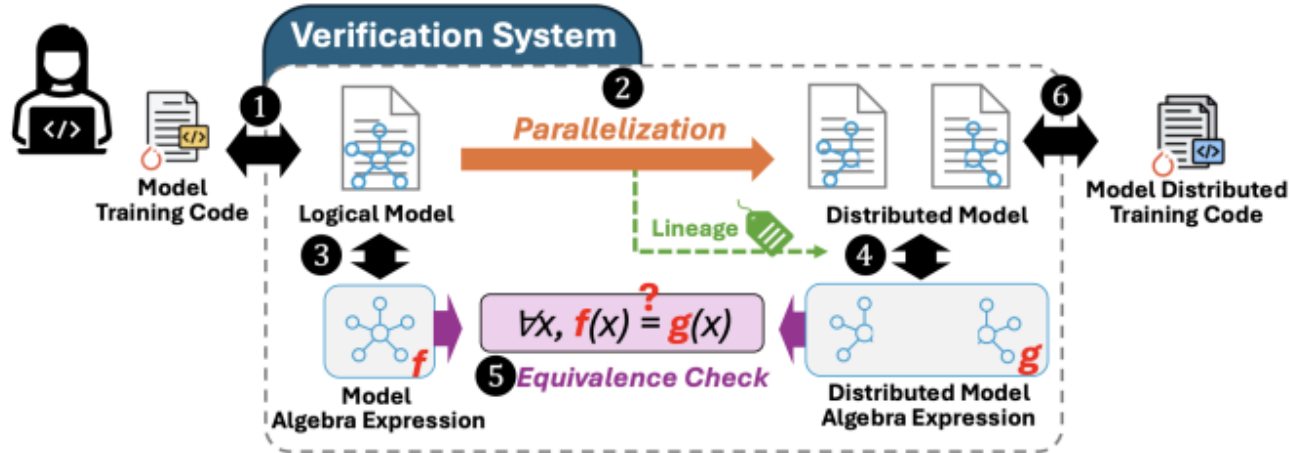


Figure 3. TRAINVERIFY System Overview.



Python

z3-solver

IRGraph

nnScaler

Results

Exp. ID	Model	Layers	DP	TP	PP	NM
L1	Llama3-8B	32	512	1	1	1
L2	Llama3-70B	80	16	8	4	32
L3	Llama3-405B	126	64	8	16	16
D1	DS-V3-16B	27	16	4	2	16
D2	DS-V3-236B	60	16	8	4	16
D3	DS-V3-671B	61	32	8	8	16

Table 2. Evaluated real-world large models.

	L1	L2	L3	D1	D2	D3
Solver Parallelism	30	30	4	30	16	8
End-to-end Time	0.5h	7.5h	47h	0.5h	3.5h	31h

Table 3. Verification time for the evaluated models.

- Successful verification of execution plans for LLaMA3 & DeepSeek-V3
- Verification finishes within 0.5 to a few hours for small & medium models
 - Up to 47 hours for largest models
- Uncovered two classes of errors in nnScaler
 - Dangling tensors & sharding a non-partitionable dimension

Results on different classes of bugs

01

Incorrect communication operators.

04

Incorrect scaling of distributed states.

02

Incorrect device assignment.

05

Incorrect pipeline scheduling.

03

Incorrect partitioning of computational operators.

06

Buffer-related bugs.

Solution Analysis

Strengths

- Mathematically verifies the correctness of an execution plan, rather than just against a few examples
- Can scale to support SOTA models with billions of parameters
- Can identify many categories of bugs

Limitations

- Reliance on graph-based execution plans & tensor lineage
- Manual specification of shape reduction rules and operator rewrites
- Doesn't verify operator implementation details beyond their semantics, the runtimes of execution plans, and certain classes of bugs



02

Oobleck

Problem

Model Scaling

- Trillion-parameter models require hundreds of TBs of space
- Models are too small for single GPUs

Solution: Must use hybrid parallelism (data + pipeline) across multiple machines.

Failures

- Failures increase linearly with number of machines
- Participating devices idle till failure recovery

Solution: Resilient yet parallel training practices.

Previous Work

Varuna

- Checkpointing states on disk
- On failure → Restart device, read disk (billions of bytes), copy state
- Extremely slow restarts

Takeaway: Need to enable quick failure handling through replicated state on memory.

Bamboo

- Compute same math across different pipelines
- Constant double overhead
- Double memory usage (frequent OOM)

Solution: Minimize redundant computation with data parallelism.

What is Oobleck?

High training
throughput

Guaranteed fault
tolerance

Fault-tolerant hybrid-parallel training framework.

Fast recovery without
additional overhead

System Components



Pipeline Template Generator

Generates a set of heterogeneous pipeline templates that can be used by the execution engine for pipeline instantiation. Pipeline templates are created only once and never change during the entire training.



Distributed Execution Engine

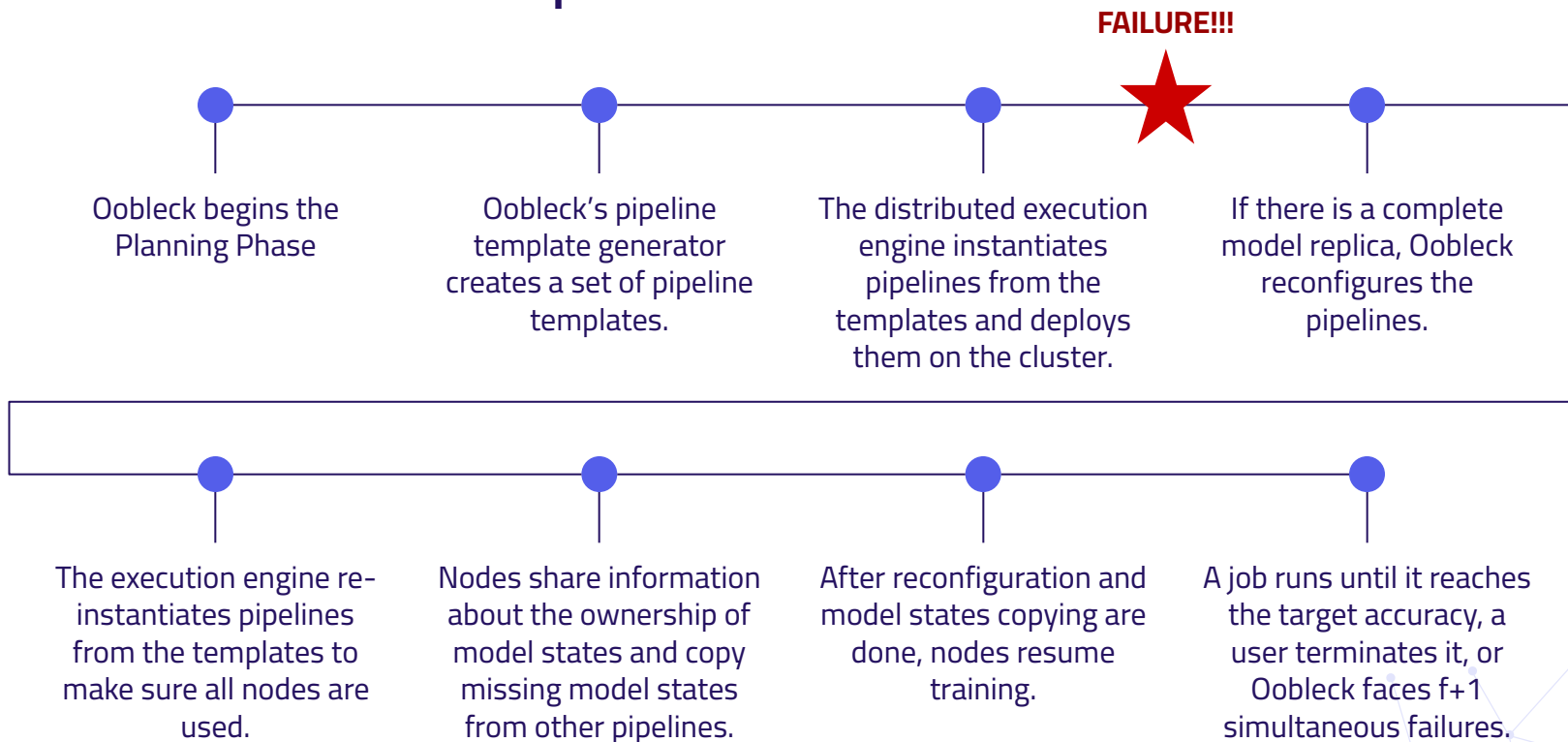
Enables efficient heterogeneous pipeline execution. Creates at least $f+1$ pipeline replicas so that at least one copy of the model exists anytime during training to handle up to f simultaneous failures.



Node Change Monitor

Detects node failures and node additions with a CPU process. Uses TCP connections between nodes as a way to detect failures.

Oobleck Example Run



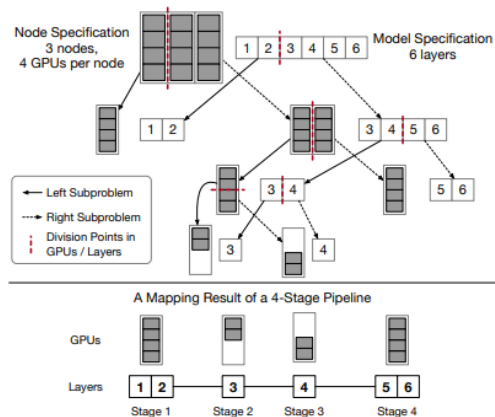
What are pipeline templates?

A pipeline is a configuration of machines that run one instance of the model. A template is a predefined configuration of the model across a set number of machines.

0.1% of Total Training
Time

Supported by Frobenius
Problem

Divide & Conquer GPU Mapping



Failure Recovery

01

Failure detection

A CPU process on each node establishes a TCP connection to a centralized CPU process. When a node dies, a socket disconnection event is triggered and broadcasted for reconfiguration.

02

Pipeline reinstantiation

The execution engine re-instantiates pipelines from the predefined pipeline templates to make sure all nodes are used. Failed pipeline gets reconfigured to a different template and starts over next batch.

03

Batch redistribution

Distributes failed batch into microbatches in order to finish iteration at same time while re-instantiated pipeline recovers.

Pipeline Reinstantiation

01

Simple reinstantiation

For each pipeline, Oobleck first checks if there is an instantiable pipeline template with remaining nodes. If so, Oobleck simply reinstantiates it and replaces the old one.

02

Borrowing nodes

Otherwise, Oobleck tries to borrow nodes from other pipelines until we have enough nodes to instantiate the smallest pipeline template. Pipelines that yield their nodes should also be reinstantiated with fewer nodes.

03

Merging pipelines

After many reconfigurations and node borrowings, all pipelines may not be able to yield their nodes. When failures happen at this point, Oobleck merges pipelines to create a bigger pipeline.





PyTorch

FSDP

HuggingFace Transformers

Merak

DeepSpeed

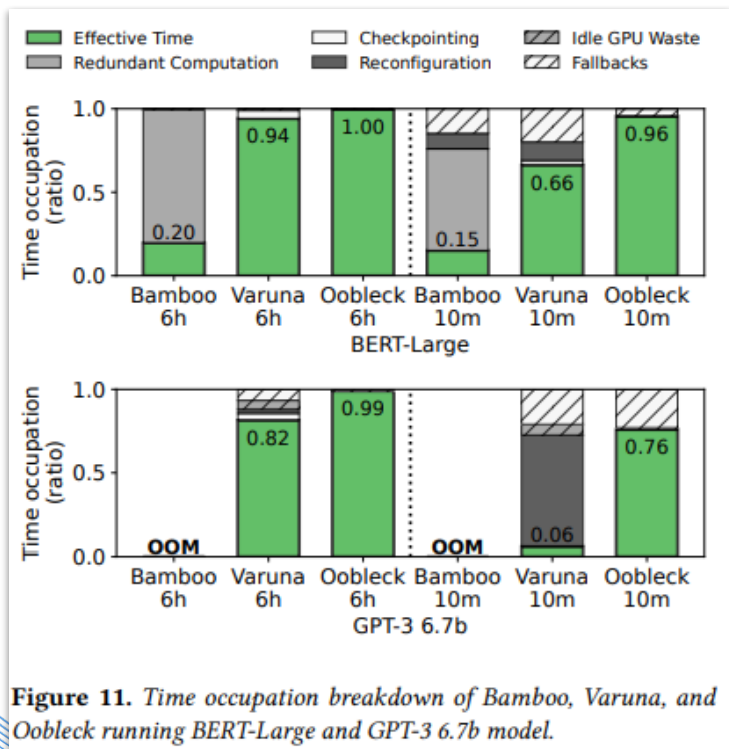
Results

Table 2. *Throughput (samples/s) with different frequency of failures. Bamboo was not able to run any GPT-3 model due to lack of memory (OOM).*

Models	BERT-Large			GPT-2			GPT-3 Medium			GPT-3 2.7b			GPT-3 6.7b		
Failure Frequency	6h	1h	10m	6h	1h	10m	6h	1h	10m	6h	1h	10m	6h	1h	10m
Bamboo	77.04	75.60	69.84	17.47	17.13	16.01									
Varuna	259.57	245.39	168.15	86.42	83.94	69.67	29.52	27.61	20.71	7.27	6.41	0.36	4.02	2.91	0.12
Oobleck	287.10	286.28	282.11	85.59	85.42	84.80	29.30	29.21	28.70	7.29	7.23	6.89	4.33	4.22	3.55

- Successful training of many large models without OOM or other system errors
- Up to 29.6x better performance in high failure conditions
- Oobleck also maintains high throughput in EC2 / GCP conditions
 - Great for LLM Training Accessibility

Results



- Oobleck consistently stays in effective use state for most cases
- Minimum fallbacks
- Removes idle waste, redundant computation, and checkpointing
 - Parallel fault tolerance guaranteed

Solution Analysis

Strengths

- Oobleck provides a mathematical guarantee that training can continue without a full restart.
- Memoization and divide-and-conquer enable the framework to handle massive models with negligible precomputation time.
- Detecting failures via TCP allows the system to bypass NCCL communication hangs.

Limitations

- Maintaining $f+1$ replicas requires significantly more total cluster memory.
- Total number of communication calls could become a bottleneck on slower interconnects.
- If a cluster experiences a massive surge in failures exceeding f , Oobleck must fall back to a standard checkpoint restart.



Thank You!